

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών

3η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614
Φίλιππος Γιαννακόπουλος 03121629

Ζήτημα 3.1

Το Ζήτημα 3.1 βρίσκεται στο αρχείο `erg3.1.asm`.

Για τον υπολογισμό της τιμής του Prescaler ($TOP=0x00FF=255$ για Fast PWM 8bit):

$$f_{PWM} = \frac{f_{clk}}{N \cdot (1 + TOP)}$$
$$N = \frac{f_{clk}}{f_{PWM} \cdot (1 + TOP)}$$
$$N = \frac{16MHz}{62500Hz \cdot (1 + 255)}$$
$$N = 1$$

Ο υπολογισμός των 13 θέσεων φωτεινότητας (duty cycle) έγινε απλά πέρνοντας το ανάλογο ποσοστό κάθε φορά του 255 και στρογγυλοποιώντας το στον πλησιέστερο ακέραιο.

```
.include "m328PBdef.inc"

.def DC_VALUE=r20
.def TablePointer=r21

.equ FOSC_MHZ=16
.equ DEL_mS=10
.equ F1=FOSC_MHZ*DEL_mS

.org 0x0
    rjmp reset

DCTable: .DB 5, 26, 46, 66, 87, 108, 128, 148, 168, 189, 209, 230, 250, 0

reset:
    ; Initialize Stack Pointer
    ldi r24, LOW(RAMEND)
    out SPL, r24
    ldi r24, HIGH(RAMEND)
    out SPH, r24

    ; Set TMR1A in Fast PWM 8bit mode with non-inverted output to PB1
    ldi r24, (0<<WGM11) | (1<<WGM10) | (0<<COM1A0) | (1<<COM1A1)
    sts TCCR1A, r24

    ; Also load CLK configuration
    ldi r24, (0<<WGM13) | (1<<WGM12) | (0<<CS12) | (0<<CS11) | (1<<CS10)
    sts TCCR1B, r24

    ; Set PORTB outputs
    ldi r24, 0b00111111
    out DDRB, r24

    ; Set PORTD as input
    clr r24
    out DDRD, r24

    ; Set r16 as a zero register for adding with carry
    ldi r16, 0

    ; Initial DC_VALUE
    ldi TablePointer, 6
    ldi ZL, LOW(DCTable*2)    ; Possible that it needs DCTable*2
    ldi ZH, HIGH(DCTable*2)
```

```

    add ZL, TablePointer
    adc ZH, r16
    lpm DC_VALUE, Z

    ; Write DC_VALUE to OCR1A
    sts OCR1AL, DC_VALUE

main:
    ; Read PORTD
    in r24, PIND

    ; If PD3 is pressed (PD3=0), handle increase
    sbrc r24, 3
    rjmp handleInc

    ; If PD4 is pressed (PD4=0), handle decrease
    sbrc r24, 4
    rjmp handleDec

    rjmp main

handleInc:
    rcall wait_x_msec
    in r24, PIND

    cpi r24, 0xFF
    brne handleInc

    cpi TablePointer, 12
    breq main

    inc TablePointer

    ldi ZL, LOW(DCtable*2)
    ldi ZH, HIGH(DCtable*2)
    add ZL, TablePointer
    adc ZH, r16
    lpm DC_VALUE, Z

    sts OCR1AL, DC_VALUE

    rjmp main

handleDec:
    rcall wait_x_msec
    in r24, PIND

    cpi r24, 0xFF
    brne handleDec

    cpi TablePointer, 0
    breq main

    dec TablePointer

    ldi ZL, LOW(DCtable*2)
    ldi ZH, HIGH(DCtable*2)
    add ZL, TablePointer
    adc ZH, r16
    lpm DC_VALUE, Z

```

```

    sts OCR1AL, DC_VALUE

    rjmp main

wait_x_msec:
    push r23
    push r24
    push r25
repeat_x:
    rcall wait_one_msec
    sbiw r24,1
    brne repeat_x

    pop r25
    pop r24
    pop r23
    ret

wait_one_msec:
    ldi r23, 247
repeat_one:
    dec r23
    nop
    brne repeat_one

    nop
    ret

```

Ζήτημα 3.2

Το Ζήτημα 3.2 βρίσκεται στο αρχείο erg3.2.c.

Η λογική επεξεργασίας και χρήσης του ADC ακολουθεί την λογική της 3.1.

Για τον υπολογισμό της μέσης τιμής των τελευταίων 16 διαδοχικών μετρήσεων του ADC, γίνεται χρήση ενός Circular Buffer και μιάς μεταβλητής sum η οποία διατηρεί το άθροισμα όλων των στοιχείων του buffer. Σε μία μέτρηση, αφαιρείτε η τελευταία θέση του πίνακα από το sum, προστίθεται η νέα μέτρηση, παίρνει την θέση του πίνακα η νέα τιμή και προχωρά κατα μία θέση ο δείκτης.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

const unsigned char DC_VALUE[] = { 5, 26, 46, 66, 87, 108, 128, 148, 168, 189, 209, 230, 250 };
unsigned int PREV[16] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
unsigned char PREV_INDEX = 0;
unsigned int sum = 0;

ISR(TIMER3_COMPA_vect) {
    ADCSRA |= (1<<ADSC);
    while ((ADCSRA & (1<<ADSC)) != 0);

    unsigned int read = ADC;

    sum += read;
    sum -= PREV[PREV_INDEX];
    PREV[PREV_INDEX] = read;
    PREV_INDEX++;
    if (PREV_INDEX == 16)
        PREV_INDEX = 0;

    read = sum>>4;

    unsigned char temp = 0b11000000;

    if (read >= 0 && read <= 200) {
        temp |= 1;
    }
    if (read >= 200 && read <= 400) {
        temp |= 1<<1;
    }
    if (read >= 400 && read <= 600) {
        temp |= 1<<2;
    }
    if (read >= 600 && read <= 800) {
        temp |= 1<<3;
    }
    if (read >= 800) {
        temp |= 1<<4;
    }

    PORTD = temp;
}

int main(void) {
    unsigned char index = 6;

    // Set TMR1A in Fast PWM 8bit mode with non-inverted output to PB1
    TCCR1A = (0<<WGM11) | (1<<WGM10) | (0<<COM1A0) | (1<<COM1A1);
```

```

// Also load CLK/1
TCCR1B = (0<<WGM13) | (1<<WGM12) | (0<<CS12) | (0<<CS11) | (1<<CS10);
// Initial value at 50%
OCR1AL = DC_VALUE[index];

// Timer 3 for interrupts to get DC data (CLK/64)
TCCR3A = 0;
TCCR3B = (0<<WGM33) | (1<<WGM32) | (0<<CS32) | (1<<CS31) | (1<<CS30);
// 16MHz / (64 * 10Hz) - 1
OCR3A = 24999;
TIMSK3 = (1<<OCIE3A);

DDRB |= 0b00111111;

// set PORTD to an output
DDRD = 0b00111111;
PORTD = 0b11000000;

// Setup ADC
ADMUX = (1<<REFS0) | (1<<MUX0);
ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

sei();

while (1) {
    unsigned char temp = PIND;
    if (!(temp & (1<<PIND6))) {
        if (index < 12) {
            index++;
            OCR1AL = DC_VALUE[index];
        }

        while (!(PIND & (1<<PIND6))) {
            _delay_ms(10);
        }
    } else if (!(temp & (1<<PIND7))) {
        if (index > 0) {
            index--;
            OCR1AL = DC_VALUE[index];
        }

        while (!(PIND & (1<<PIND7))) {
            _delay_ms(10);
        }
    }
}
}

```

Ζήτημα 3.3

Το Ζήτημα 3.3 βρίσκεται στο αρχείο erg3.3.c.

Παρόμοια λογική με την 3.1.

Για την χρήση του ADC ως ρυθμιστή της φωτεινότητας (και άρα του Duty Cycle), κάνουμε απλά right shift 2 φορές το 10bit αποτέλεσμα του και το βάζουμε στον OCR1AL.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

const unsigned char DC_VALUE[] = { 5, 26, 46, 66, 87, 108, 128, 148, 168, 189, 209, 230, 250 };

int main(void) {
    unsigned char index = 6;
    unsigned char status = 0x00;
    unsigned char temp;

    // Set TMR1A in Fast PWM 8bit mode with non-inverted output to PB1
    TCCR1A = (0<<WGM11) | (1<<WGM10) | (0<<COM1A0) | (1<<COM1A1);
    // Also load CLK/1
    TCCR1B = (0<<WGM13) | (1<<WGM12) | (0<<CS12) | (0<<CS11) | (1<<CS10);
    // Initial value at 50%
    OCR1AL = DC_VALUE[index];

    DDRB |= 0b00111111;

    // set PORTD to an input
    DDRD = 0x00;
    PORTD = 0xFF;

    // Setup ADC
    ADMUX = (1<<REFS0);
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

    sei();

    while (1) {
        if (status == 0xFF) {
            ADCSRA |= (1<<ADSC);
            while ((ADCSRA & (1<<ADSC)) != 0);

            unsigned int read = ADC>>2;
            OCR1AL = read;
        } else {
            temp = PIND;
            if (!(temp & (1<<PIND3))) {
                if (index < 12) {
                    index++;
                    OCR1AL = DC_VALUE[index];
                }
                while (!(PIND & (1<<PIND3))) {
                    _delay_ms(10);
                }
            } else if (!(temp & (1<<PIND4))) {
                if (index > 0) {
                    index--;
                    OCR1AL = DC_VALUE[index];
                }
            }
        }
    }
}
```

```

        while (!(PIND & (1<<PIND4))) {
            _delay_ms(10);
        }
    }
    temp = PIND;
    if (!(temp & (1<<PIND6))) {
        status = 0x00;
        OCR1A = DC_VALUE[index];
        while (!(PIND & (1<<PIND6))) {
            _delay_ms(10);
        }
    } else if (!(temp & (1<<PIND7))) {
        status = 0xFF;
        while (!(PIND & (1<<PIND7))) {
            _delay_ms(10);
        }
    }
}
}
}

```